

AAMT-WP-13: Inference on Consumer Silicon

AAMT-WP-13 | AsAManThinks Research | May 2026 | DOI:
10.5281/zenodo.19600795

Inference on Consumer Silicon: Phase-Gated Compute, Shared-Base LoRA Sharding, and Process-Isolated MPS Execution for 2B-Parameter Models

Weslyn Whitehead Jr.

AsAManThinks Research, Berkeley, CA, USA |
weslyn@asamanthinks.com

ORCID: [to be registered](#) | License: CC BY 4.0

Inference on Consumer Silicon: Phase-Gated Compute, Shared-Base LoRA Sharding, and Process-Isolated MPS Execution for 2B-Parameter Models

AsAManThinks Research — Working Paper WP-13

Weslyn Whitehead Jr. (Docwes / Yare Ace NoK)

ORCID: <https://orcid.org/0009-0005-7707-3210>

May 2026 • MaiiaM Alchemist v0.4.1

DOI: 10.5281/zenodo.19600795

Abstract

We present a system for running 2B-parameter mixture-of-experts language models on Apple Silicon consumer hardware (M-series Macs) with sub-200ms time-to-first-token (TTFT), 38-61 tokens-per-second (TPS) throughput, and peak RSS under 6 GB. The system combines five techniques: (1) **phase-gated layer depth** — PTPS breath scheduling (WP-03) dynamically reduces active residual block depth to 25-100% based on generation phase; (2) **semantic KV cache flushing** — cache retention drops to 50% at EXHALE and 0% at EMPTY, based on generation phase rather than attention scores; (3) **zero-parameter symbolic routing** — the VortexGate (WP-01) replaces a ~8M-parameter learned gate with a 4D linear projection (8,196 params), reducing VRAM pressure; (4) **shared-base LoRA shard serving** — nine expert adapters (1-5 MB each) share a single 2B base model in memory, with only top-k adapters merged per token; and (5) **MPS process isolation** — the PyTorch sidecar runs in a dedicated Python process, preventing Electron's V8 heap from competing for Apple Silicon's unified memory pool. Performance is validated by a 22-suite eval harness (14 core + 8 Unity suites) with real statistical comparison (Cohen's d, paired t-tests) across runs.

1. Introduction

Series note: This paper is part of the AAMT Working Paper Series. This paper is a systems integration paper — it combines techniques from WP-01 (VG-MoE, zero-parameter routing), WP-02 (HCRS, HeartScale alignment), WP-03 (PTPS, phase-gated scheduling), and WP-08 (MSI, shared-base LoRA ensembles). Readers are directed to those papers for full derivations of each technique; this paper focuses on how they combine and how their joint performance is measured on consumer hardware.

Consumer Apple Silicon (M1-M4) is an attractive target for personal AI model deployment. The unified memory architecture eliminates PCIe bandwidth as a bottleneck, Neural Engine acceleration is available, and 32-96 GB unified memory variants can hold a 2B-parameter model comfortably. However, the same unified memory pool is shared by all processes, including the UI framework (Electron/V8), making naive deployment — loading the model in the same process as the UI — non-viable for interactive use.

This paper describes the engineering decisions in MaiaM Alchemist v0.4.1 that together achieve interactive inference rates on a single M-series Mac, and the eval infrastructure used to measure and gate regressions.

2. Technique 1: Phase-Gated Layer Depth Reduction

2.1 The compute budget table

From the PTPS scheduler (WP-03), each breath phase maps to a `ComputeBudget` :

Phase	layer_depth_frac	top_k_attention	kv_cache_frac
INHALE	1.0	8/8	1.0
HOLD	0.75	4/8	1.0
EXHALE	1.0	6/8	0.5
EMPTY	0.25	2/8	0.0

Averaged over one complete cycle: layer_depth = 0.75, attention = 5/8, kv_cache = 0.625.

2.2 Layer skipping implementation

At HOLD phase, the forward pass skips every 4th residual block (layer_depth_frac = 0.75). At EMPTY phase, only the first and last 12.5% of blocks run (layer_depth_frac = 0.25). This is implemented as a dynamic layer mask applied before the forward pass, not as a model architecture change. The same weights are used; some layers simply don't execute.

2.3 Measured impact on TTFT

The `LatencyThroughputSuite` measures p50/p95/p99 TTFT across 50 prompts. Phase-gated depth primarily reduces TTFT for EMPTY and HOLD phase tokens (which represent ~40% of generation by the

token-clock schedule). The overall p95 TTFT improvement over fixed full-depth baseline is approximately 10%.

3. Technique 2: Semantic KV Cache Flushing

3.1 Distinction from attention-score compression

Existing KV cache compression methods (H2O, SnapKV, StreamingLLM) decide which KV entries to evict based on attention score history — entries that receive little attention are considered less important. These methods are reactive: they respond to the model’s computed attention patterns.

Semantic KV cache flushing is **proactive and phase-driven**: at EXHALE phase, the cache is truncated to 50% regardless of attention scores. At EMPTY phase, the cache is cleared entirely. The rationale is semantic rather than statistical: at EXHALE, the model has committed to a generation direction, and the early-context KV entries are genuinely less relevant to the remaining tokens — not because they received low attention, but because the generation arc has progressed past them.

3.2 Why this is safe

The HeartScale sampler (WP-02) is gating every token for archetype coherence. A token that would cause a large archetype drift — indicating that early context is still strongly constraining generation — will be resampled. The HeartScale gate provides a soft guarantee that EXHALE-phase tokens are low-drift, meaning flushing old context is semantically safe at exactly the phase when the cache flush occurs.

3.3 Memory impact

For a 2B model with a 2048-token context window, the KV cache occupies approximately 512 MB at BF16 precision. Flushing to 0% at EMPTY phase and 50% at EXHALE frees an average of ~200 MB per generation across a full breath cycle. On Apple Silicon’s 16-32 GB unified memory configurations, this is meaningful headroom for the UI and other processes.

4. Technique 3: Zero-Parameter Symbolic Routing

A standard MoE gate at $D=2048$ hidden dimension is a `Linear(2048 → N_experts)` followed by softmax-topk. For $N=9$ experts, this is $2048 \times 9 + 9 = 18,441$ parameters plus the learned routing statistics maintained during training.

The VortexGate is `Linear(2048 → 4) → sigmoid → vortex_project()`. Parameters: $2048 \times 4 + 4 = 8,196$. The `vortex_project()` function adds zero parameters. Gradient computation through the gate is cheaper by the same factor (4 output dims vs 9). During inference, the gate produces a 16-element Meji distribution that is collapsed to expert weights via a constant 16×9 matrix multiplication — no learned softmax, no temperature scaling, no auxiliary loss state.

For models where the gate is called at every forward pass (once per token), the parameter count reduction of 55% for the gate module translates directly to reduced memory bandwidth during weight loading and gradient accumulation during training.

5. Technique 4: Shared-Base LoRA Shard Serving

5.1 Architecture

The base model (`google/gemma-2-2b` , ~2.6B parameters, ~5.2 GB at BF16) is loaded once and kept in MPS device memory. Nine LoRA adapters are maintained as CPU-resident weight deltas, each 1-5 MB. At inference time:

1. VortexGate computes expert weights for the current token.
2. Top-k expert adapter delta weights are merged into the base weight tensors (additive merge: $W_{\text{effective}} = W_{\text{base}} + \alpha \cdot W_{\text{lor}_A} \cdot W_{\text{lor}_B}$).
3. Forward pass executes through the merged weight.
4. Merge is undone before the next token (or the merged weight is cached for the next token if the same top-k set is selected again).

5.2 Memory accounting

Component	Size	Location
Base model (BF16)	5.2 GB	MPS device memory
9 LoRA adapters (BF16)	~36 MB total	CPU RAM
KV cache (2048 ctx, BF16)	~512 MB	MPS device memory
Eval harness + Python runtime	~400 MB	CPU RAM
Electron + UI	~300 MB	CPU RAM
Total	~6.4 GB	

This is well within a 16 GB Apple Silicon machine. Compare to a naive 9-model ensemble (9 × 5.2 GB = 46.8 GB) or a single full-precision model (10.4 GB).

5.3 The Multiverse variant

When N adapters are trained with distinct chaos seeds (T2.3 in the tier features), the same base stays resident while N adapter delta sets are blended in logit space. N=4 requires 4 × ~4 MB = ~16 MB additional CPU RAM — negligible — while providing ensemble diversity without any additional VRAM cost.

6. Technique 5: MPS Process Isolation

6.1 The unified memory contention problem

Apple Silicon uses a unified memory architecture: CPU, GPU (MPS), and Neural Engine share the same physical DRAM. The Electron runtime (V8 JavaScript engine + Chromium renderer) aggressively allocates heap memory. On a 16 GB machine running Electron + Python PyTorch simultaneously, memory pressure causes the OS to page MPS-resident tensors to swap, destroying inference throughput.

6.2 The sidecar solution

The PyTorch runtime runs in a dedicated Python process (`python -m harmonic_engine.sidecar`) with no memory-sharing with the Electron process. Communication is via stdio JSON-RPC — character streams, not shared memory. The Electron main process spawns the sidecar, multiplexes JSON-RPC calls over its stdin/stdout, and manages restarts.

The separation has two effects: (1) the OS can page-lock MPS-resident tensors for the Python process without V8's allocation pressure triggering page reclaim; (2) a Python crash (OOM, CUDA error) does not crash the UI — the `SidecarManager` detects the exit and respawns with exponential backoff (1s → 2s → 4s → 8s → 16s cap).

6.3 Path augmentation for cross-tool coordination

The sidecar and eval suite tools (`dotnet`, `glslangValidator`, `eslint`, Unity CLI) must be discoverable from the spawned process's `PATH`. The `augmentedPath()` method builds a `PATH` string that includes Homebrew, NVM, `~/.local/bin` (uv default), and other common developer tool locations:

```
augmentedPath() {
  const extras = [
    `${home}/.local/bin`,    // uv
    '/opt/homebrew/bin',    // Homebrew (Apple Silicon)
    '/opt/homebrew/sbin',
    '/usr/local/bin',       // Homebrew (Intel)
    `${home}/.nvm/versions/...`,
  ];
  return [...extras, process.env.PATH].join(':');
}
```

This ensures tools are found in spawned processes regardless of whether the user's shell profile has been sourced.

7. Empirical Validation

7.1 Eval suite architecture

The 22-suite eval harness (`packages/eval-suite`) is integrated into the sidecar via the `eval.*` JSON-RPC namespace. The harness includes:

Compute-sensitive suites (run exclusively): - `latency_throughput`: 50 prompts, measures p50/p95/p99 TTFT and TPS - `memory_footprint`: 20 prompts, 50ms RSS sampling thread, peak RSS tracking - `breath_phase_coherence`: 30 prompts, per-phase TPS vs. budget scoring

Quality suites (run in parallel, max 3 workers): - `harmonic_coherence`: HeartScale yoked-rate + vortex tightness per archetype - `archetype_alignment`, `archetype_voicing`: alignment-specific quality - `heartscale_violations`: adversarial prompts, false-accept rate - `csharp_compile_rate`, `shader_compile_rate`, `unity_api_correctness`: domain quality - Standard: `mmlu_subset`, `gsm8k_lite`, `humaneval_lite`, `instruction_following`

7.2 Run-over-run comparison

The `compare.py` module computes statistical significance for every metric delta:

```
res = scipy_stats.ttest_rel(base_samples[:n], cand_samples[:n])
p_value = float(res.pvalue)
significant = p_value < 0.05
cohen_d = _cohen_d(base_samples, cand_samples)
```

Cohen’s *d* provides effect size context beyond p-values — a statistically significant result with `cohen_d < 0.1` may not represent a practically meaningful improvement. The `compare.py` module surfaces Cohen’s *d* alongside the p-value so that the operator can make an informed promotion decision. For example, an operator might adopt a convention of requiring `cohen_d > 0.2` (Cohen’s small-effect threshold) as a minimum before claiming a meaningful throughput improvement, preventing noise-driven promotions. The promotion gate itself is defined in WP-04 (LACL) based on metric regression bounds, not a Cohen’s *d* hard cutoff.

7.3 Observed throughput (M2 Pro, 32 GB)

Configuration	TPS (mean)	TTFT p95 (ms)	Peak RSS (MB)
Fixed full budget	38	185	5,820
PTPS phase-gated	52	162	5,820
+ semantic KV flush	54	158	5,610
+ speculative decode (EXHALE/EMPTY)	61	158	5,680

The `BreathPhaseCoherenceSuite` confirms phase-specific TPS targets are met: INHALE tokens run slower (full depth, no spec decode), EXHALE tokens run faster (speculative decode active, KV flushed). The asymmetric scoring (undershoot penalized for INHALE/HOLD, overshoot penalized for EMPTY) ensures the scheduler meets its per-phase contracts.

8. Limitations

1. The layer-skipping implementation requires the base model to support dynamic layer masking, which Gemma 2 does via its standard `skip_layers` argument. Models without this support require a custom forward pass patch.
2. The shared-base LoRA merge adds one matmul per active adapter per layer per token. For $k=2$ active adapters and 26 layers, this is 52 additional matmuls per token — acceptable for 2B models but increasingly costly for larger bases.
3. Semantic KV flushing at EMPTY phase can cause regeneration artifacts if EMPTY tokens are mid-sentence rather than terminal. The HeartScale gate reduces this risk but does not eliminate it for unusually long EMPTY phases.

9. Conclusion

Five techniques — phase-gated depth, semantic KV cache flushing, zero-parameter symbolic routing, shared-base LoRA sharding, and MPS process isolation — together enable interactive 2B-parameter MoE inference on a single consumer Apple Silicon Mac. The combination is validated by a 22-suite eval harness with statistical significance testing. No single technique is sufficient; the techniques are complementary and their memory and compute savings are approximately additive.

References

- Leviathan, Y. et al. (2023). Fast inference from transformers via speculative decoding.
- Zhang, Z. et al. (2023). H2O: Heavy-Hitter Oracle for efficient generative inference of large language models.
- Xiao, G. et al. (2024). StreamingLLM: Efficient streaming language models with attention sinks.
- Li, Y. et al. (2024). SnapKV: LLM knows what you are looking for before generation.
- Schuster, T. et al. (2022). Confident adaptive language modeling.
- Hu, E. et al. (2021). LoRA: Low-rank adaptation of large language models.
- Whitehead, W. (2026). MaiiaM Wings. Zenodo. DOI: 10.5281/zenodo.19600795